

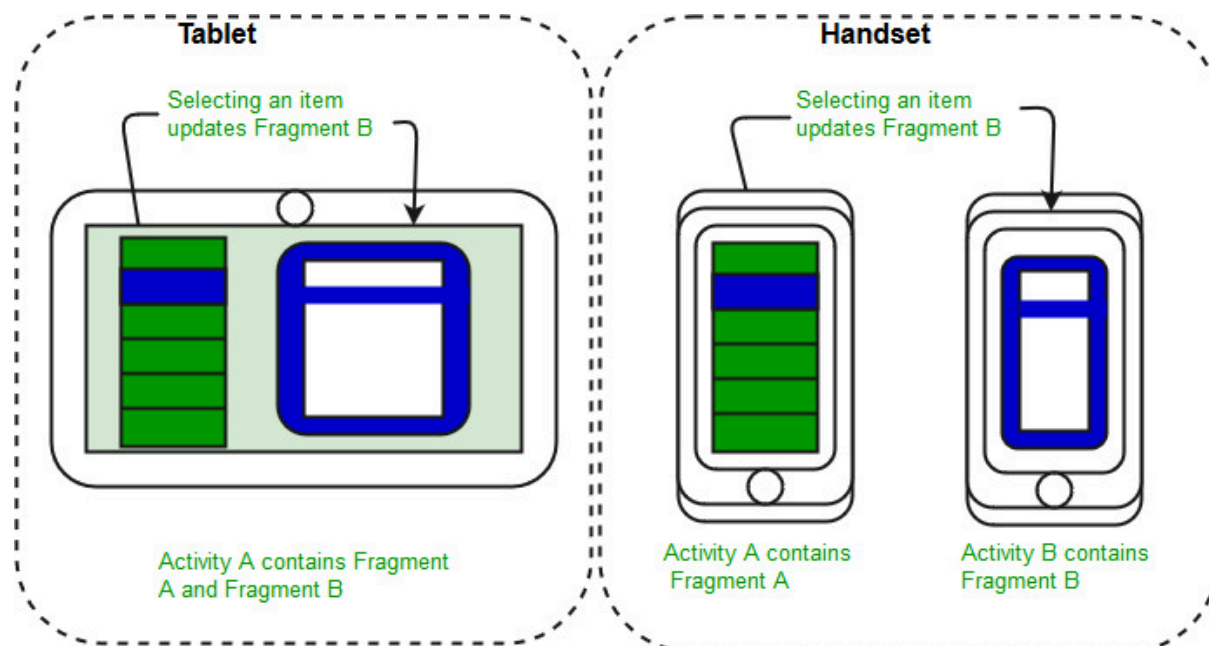
Fragments, Toolbar y ViewPager2

Fragments

Un fragmento es una parte de una actividad que permite un diseño de actividad más modular. Un fragmento encapsula la funcionalidad para que sea más fácil de reutilizar dentro de actividades y diseños. Los dispositivos Android existen en una variedad de tamaños y densidades de pantalla. Los fragmentos simplifican la reutilización de componentes en diferentes diseños y su lógica. Puede crear diseños de panel único para teléfonos y diseños de paneles múltiples para tabletas. También puede utilizar fragmentos para admitir diferentes diseños de orientación horizontal y vertical en un teléfono inteligente.

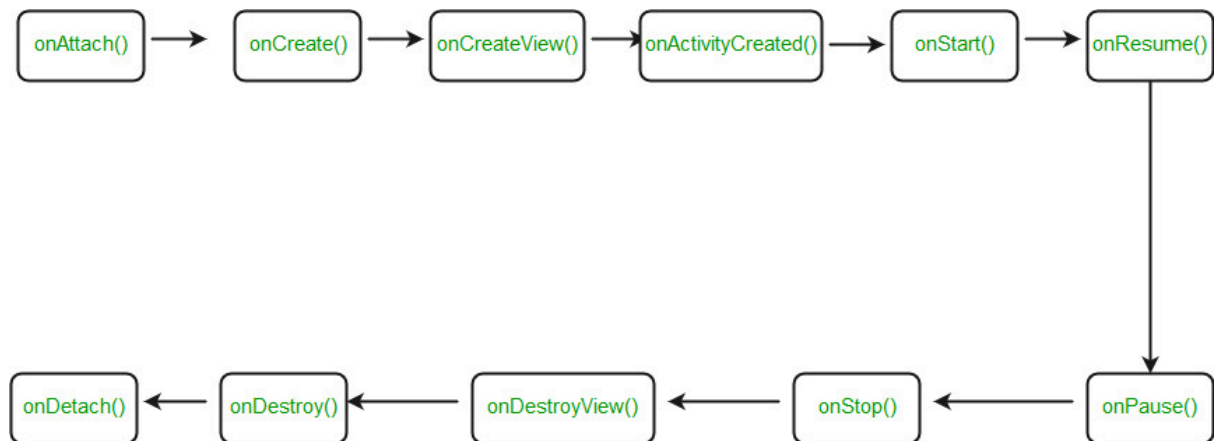
Un fragment no puede considerarse ni un control ni un contenedor, aunque se parecería más a lo segundo. Un fragment es una porción de la interfaz de usuario que puede añadirse o eliminarse de la interfaz de forma independiente al resto de elementos de la actividad, y que por supuesto puede reutilizarse en otras actividades. Esto nos permite poder dividir nuestra interfaz en varias porciones de forma que podamos diseñar diversas configuraciones de pantalla, dependiendo de su tamaño y orientación, sin tener que duplicar código en ningún momento, sino tan sólo utilizando o no los distintos fragmentos para cada una de las posibles configuraciones.

Los fragmentos simplifican la reutilización de componentes en diferentes diseños y su lógica. Puede construir diseños de un solo panel para teléfonos (teléfonos) y diseños multipane para tabletas.



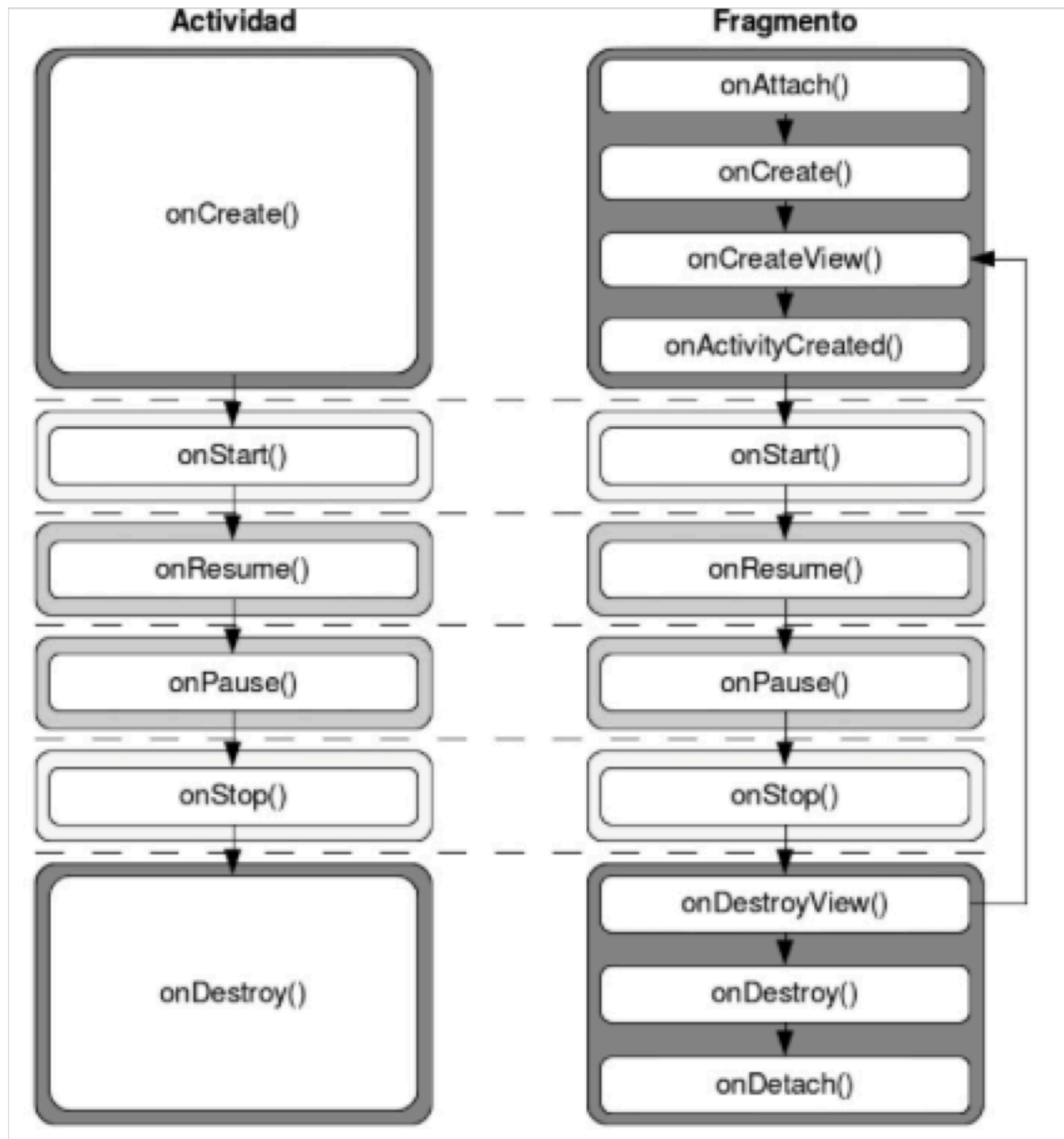
Ciclo de vida de los fragments

Como veremos más adelante, los fragments son partes de la interfaz que tienen la capacidad de mostrarse o no dependiendo de multitud de factores como pueden ser interacción con el usuario, configuraciones de la pantalla, etc... Muchos de los métodos son compartidos con las activitys, existiendo diferencia con los elementos que asocian / eliminan el fragment con la pantalla. El ciclo de vida comparado con el de las pantallas es el siguiente:



Dentro del ciclo de vida, cabe destacar los siguientes métodos:

- `onAttach`: el fragment se vincula con la actividad. Se suele utilizar para poder trabajar con el contexto, ya que es parámetro dentro del método
- `onCreate`: el fragmento está asociado pero no cuenta con interfaz gráfica. Se suele utilizar para poder instanciar elementos lógicos
- `onCreateView`: el fragmento se asocia a la parte gráfica
- `onActivityCreated`: el fragmento ya está disponible y empieza a ser visto



```
implementation(libs.androidx.fragment.ktx)
```

```
[versions]
fragmentKtx = "1.8.9"
```

```
[libraries]
androidx-fragment-ktx = { module = "androidx.fragment:fragment-ktx", version.ref = "fragmentKtx" }
```

Para utilizar fragmentos necesitamos:

- Crear una clase que extienda de Fragment
- Crear un layout para el fragmento
- Añadir el fragmento a la actividad

Añadir un fragmento de forma estática

```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tvAct"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Text view del layout del activity_main"
        app:layout_constraintBottom_toTopOf="@id/staticFragment"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/staticFragment"
        android:name="com.dam.fragmentosejemplo.FragmentoUno"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/tvAct"
        tools:layout="@layout/fragmento_uno" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

*Crear fragmento "FragmentoUno"

*Crear layout "fragmento_uno"

Un fragmento estático se usa cuando:

- Siempre necesitas ese fragmento en la actividad. No cambia durante la ejecución, y siempre debe estar presente.
- Tu diseño es simple y no necesita cambiar fragmentos dinámicamente. Ideal cuando no necesitas reemplazar ni ocultar ese fragmento en tiempo de ejecución.

- Buscas mantener una interfaz modular sin escribir lógica adicional. Puedes reutilizar el fragmento en diferentes pantallas declarando su clase directamente en el XML.

Fragmento dinámico (en tiempo de ejecución)

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/staticFragment"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toBottomOf="@id/tvAct" />
```

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Crear una instancia del fragmento
        val firstFragment = FragmentoUno()
        supportFragmentManager.beginTransaction()
            .add(R.id.staticFragment, firstFragment)
            .commit()

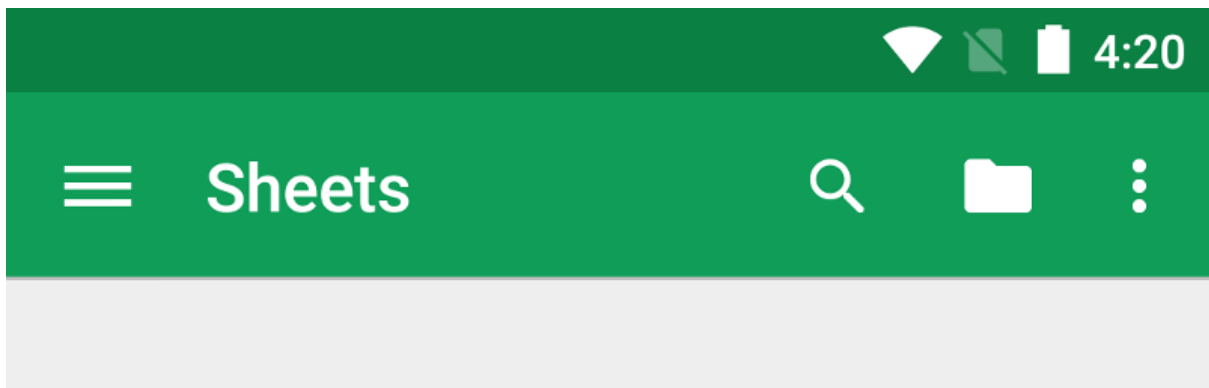
    }
}
```

.add() -> Añade un fragmento sin eliminar los anteriores. Útil cuando quieres apilar o mostrar varios

.replace() -> Reemplaza el fragmento actual. Cuando solo quieres uno a la vez

Actionbar/Appbar/Toolbar

La app bar, o también llamada action bar o tool bar, es la barra de título y herramientas que aparece en la parte superior de la gran mayoría de aplicaciones actuales de la plataforma Android.



La app bar suele mostrar el título de la app, una serie de botones de acción y/o un menú desplegable (menú de overflow) donde se incluyen más acciones que no tienen espacio para mostrarse como botón o simplemente no se quieren mostrar como tal.

Usar la barra de aplicaciones hace que tu aplicación sea consistente con el resto de aplicaciones de Android, ya que permite que aprendan a usar la aplicación rápidamente. Android proporciona este componente a través de la librería appcompat, que suele venir ya incluida por defecto en todos nuestros proyectos, la podemos encontrar en la sección dependencies del fichero build.gradle de nuestro modulo:

```
implementation(libs.androidx.appcompat)
```

Otro requisito es que tu activity extienda de AppCompatActivity, que por defecto viene incluido.

```
class MainActivity : AppCompatActivity() {  
  
    //resto del código  
  
}
```

Ahora bien, si nos vamos a nuestro fichero manifest, encontramos la siguiente línea, que especifica el tema por defecto de la aplicación

```
android:theme="@style/Theme.MenuSuperior"
```

Este tema, configurado en los archivos de estilos en res/values/styles.xml, debería heredar

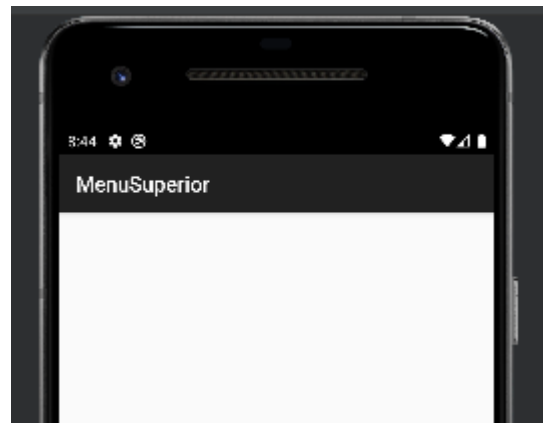
de uno de los temas de Material o AppCompat, como Theme.Material3.DayNight.NoActionBar o Theme.AppCompat.Light.DarkActionBar, que determina el estilo y el fondo de la App Bar.

Theme.Material3.DayNight.NoActionBar

Ahora nos aseguramos que el tema de nuestra aplicación extiende de alguno de los temas Theme.AppCompat disponibles, en mi caso Theme.AppCompat.Light.DarkActionBar (fondo claro con action bar oscura):

Theme.AppCompat.Light.DarkActionBar

Solo con este cambio, ya tenemos nuestra action bar en la parte superior de nuestra aplicación. Nuestra action bar por defecto solo contiene el título de nuestra aplicación, que está configurado en el manifest.



Para personalizar nuestro menú debemos de definir un fichero xml dentro de /res/menu. Dicho archivo tendrá un elemento raíz <menu> y una serie de elementos llamados <item> que representan cada una de las opciones disponibles. Cada elemento item podrá tener varios atributos, a destacar:

- **android:id**. El ID identificativo del elemento, con el que podremos hacer referencia dicha opción.
- **android:title**. El texto que se visualizará
- **android:icon**. El icono asociado
- **android:showAsAction**. Si se está mostrando una action bar, este atributo indica si la opción de menú se mostrará como botón de acción o como parte del menú de overflow. Puede tomar varios valores:
 - **ifRoom**. Se mostrará como botón de acción sólo si hay espacio disponible.
 - **withText**. Se mostrará el texto de la opción junto al icono en el caso de que éste se esté mostrando como botón de acción.
 - **never**. La opción siempre se mostrará como parte del menú de overflow.
 - **always**. La opción siempre se mostrará como botón de acción. Este valor puede provocar que los elementos se solapen si no hay espacio suficiente para ellos.

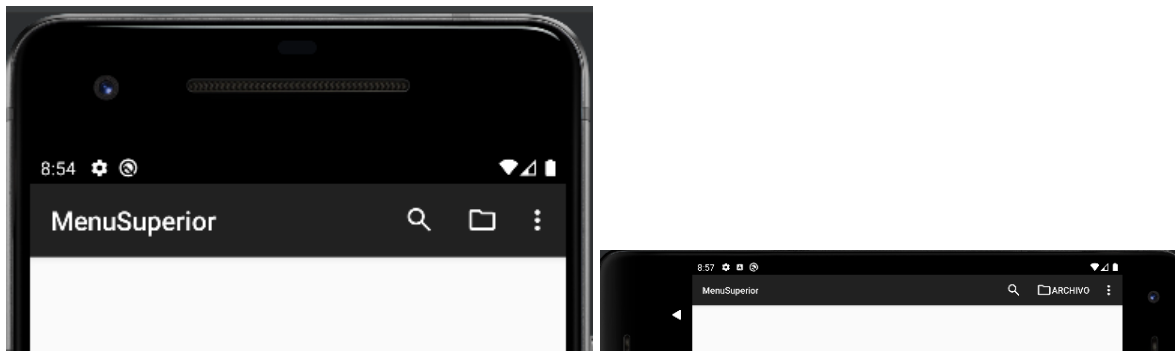
Para nuestro ejemplo crearemos:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto">
<item android:id="@+id/action_buscar"
android:title="@string/action_buscar"
android:icon="@drawable/lupa"
android:orderInCategory="100"
app:showAsAction="ifRoom" />
<item android:id="@+id/action_file"
android:title="@string/archivo"
android:icon="@drawable/file"
android:orderInCategory="100"
app:showAsAction="ifRoom|withText" />
<item android:id="@+id/action_opciones"
android:title="@string/action_opciones"
android:orderInCategory="100"
app:showAsAction="never" />
<item android:id="@+id/action_salir"
android:title="@string/action_salir"
android:orderInCategory="100"
app:showAsAction="never" />
</menu>
```

Los iconos utilizados son de <https://fonts.google.com/icons>

por último, solo nos queda asociar este menú con nuestra actividad principal, que se realiza sobreescribiendo el método `onOptionsItemSelected()` en la actividad, dentro del cual tenemos que llamar al método `inflate()` pasándole como parámetro el ID del fichero XML, por ejemplo:

```
override fun onOptionsItemSelected(menu: Menu): Boolean {
    menuInflater.inflate(R.menu.menu_main, menu)
    return true
}
```

Para configurar nuestra AppBar podemos utilizar en `/res/values/themes.xml`:

- colorPrimary. Color principal de la aplicación, como el color de fondo de la action bar.
- colorPrimaryDark. Variante más oscura del color principal, que por ejemplo se utilizará como color de la barra de estado (la barra superior que contiene los iconos de notificación, batería, reloj, ...).
- colorAccent. Suele ser el color utilizado para destacar el botón de acción principal de la aplicación (por ejemplo un botón flotante, y para otros pequeños detalles de la interfaz, como por ejemplo algunos controles, cuadros de texto o checkboxes).

Ejemplo:

```
<item name="colorPrimary">#6200EE</item>
<item name="colorPrimaryDark">#3700B3</item>
<item name="colorAccent">#03DAC5</item>
```

Ahora que ya tenemos nuestra barra, solo nos queda personalizar las acciones cuando los elementos son seleccionados. Para ello sobrescribimos el método `onOptionsItemSelected()` en nuestra actividad. Dentro de él consultaremos la opción seleccionada con `getItemId()`

```
override fun onOptionsItemSelected(item: MenuItem) = when (item.itemId) {
    R.id.action_file -> {
        Log.i("ActionBar", "Archivo!")
        true
    }
    R.id.action_buscar -> {
        Log.i("ActionBar", "Buscar!")
        true
    }
    R.id.action_opciones -> {
        Log.i("ActionBar", "Opciones!")
        true
    }
}
```

```

}
R.id.action_salir -> {
Log.i("ActionBar", "Salir!")
true
}
else -> {
super.onOptionsItemSelected(item)
}
}
}

```

Una forma más flexible y personalizable de añadir una appbar a una aplicación es utilizar el componente Toolbar también proporcionado por la librería appcompat. De esta manera, podemos incluir de forma explícita la actionBar en nuestros layouts como si se tratase de otro control. Y ya no sólo en la parte superior de la pantalla, sino también en cualquier otro lugar de la aplicación donde queramos utilizar esta funcionalidad de barra de acciones.

Al igual que el appbar por defecto, debemos de comprobar que incluimos la librería appcompat y que nuestra actividad hereda de AppCompatActivity.

Lo siguiente es desactivar la funcionalidad por defecto de inclusión de la appbar. Para ello, modificamos el fichero res/values/styles.xml para hacer que nuestro tema extienda de:

- Theme.AppCompat.NoActionBar
- Theme.AppCompat.Light.NoActionBar

Utilizaremos la versión clara (light).

```

<style name="Base.Theme.MenuSuperior" parent="Theme.AppCompat.Light.NoActionBar">
<item name="colorPrimary">@color/colorPrimary</item>
<item name="colorPrimaryDark">@color/colorPrimaryDark</item>
<item name="colorAccent">@color/colorAccent</item>
</style>

```

Donde nuestro archivo colors.xml contiene:

```

<color name="colorPrimary">#3F51B5</color>
<color name="colorPrimaryDark">#303F9F</color>
<color name="colorAccent">#FF4081</color>

```

Una vez configurado nuestro style ya podemos añadir nuestro Toolbar en el layout de la actividad. Para ello usamos el componente Toolbar:

```

<androidx.appcompat.widget.Toolbar
    android:id="@+id/toolbarPrincipal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="?attr/colorPrimary"
    android:elevation="4dp"
    android:theme="@style/ThemeOverlay.AppCompat.Dark.ActionBar"
    app:popupTheme="@style/ThemeOverlay.AppCompat.Light"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"/>

```

Le hemos añadido los constraint necesarios para que se ancle en la parte superior y además le hemos asignado las propiedades habituales como un id, el ancho y el alto.

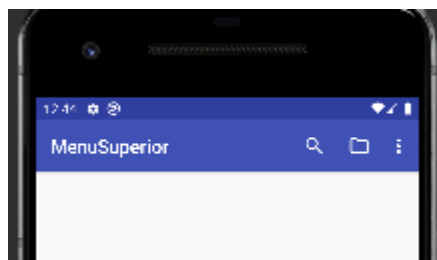
- android:background. Asignaremos a esta propiedad el valor `?attr/colorPrimary` de forma que se utilice como color de la barra de herramientas el que hemos definido como `colorPrimary` en el tema de la aplicación (archivo `/res/values/styles.xml`).
- android:elevation. Esta propiedad define la elevación del componente, lo que determina la sombra que proyectará sobre el elemento inferior. La elevación estándar de la action bar definida en las guías de diseño es de 4dp.
- android:theme. Mediante esta propiedad definimos el tema a utilizar por la Toolbar (y que heredarán sus controles hijos). No debemos confundir esto con el tema definido a nivel de aplicación en el archivo `styles.xml`. Utilizamos el tema global `Theme.AppCompat.Light.DarkActionBar` y en la Toolbar utilizaremos el tema oscuro es decir, `ThemeOverlay.AppCompat.Dark.ActionBar`.
- app:popupTheme: Para conseguir que la barra sea oscura pero el menú claro podemos asignar un tema específico sólo a este menú utilizando la propiedad `app:popupTheme`, en nuestro caso el tema `ThemeOverlay.AppCompat.Light`.

Ahora nos falta asignar en nuestra actividad que el Toolbar creado actuará como `AppBar`. En el método `onCreate()` llamamos a `setSupportActionBar()` con la referencia a la toolbar

```

setSupportActionBar(findViewById(R.id.toolbarPrincipal))

```



Llegados a este punto hemos conseguido el mismo resultado que la `AppBar` por defecto. ¿Cuáles son los beneficios de utilizar esta forma? Podemos modificar su tamaño, estilos, reciclar código...

Por ejemplo. Vamos a indicar que no nos muestre el titulo de la app justo después de `setSupportActionBar` en nuestra actividad

```
supportActionBar?.setDisplayShowTitleEnabled(false);
```

Añadimos a nuestra toolbar la propiedad `gravity` para que se enganche donde queremos, en este caso arriba.

```
android:gravity="top"
```

Y finalmente dentro de la Toolbar añadimos

```
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20dp"
        android:text="Titulo" />
    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="10dp"
        android:text="Subtitulo" />
</LinearLayout>
```

Y obtenemos como resultado:



Otra ventaja es poder usar nuestro Toolbar como componente independiente. Como podría ser un `CardView`:

```
<androidx.cardview.widget.CardView
    android:id="@+id/card_view"
    android:layout_gravity="center"
    android:layout_width="match_parent"
    android:layout_height="200dp"
```

```

app:cardUseCompatPadding="true"
app:cardCornerRadius="4dp"
android:layout_margin="8dp"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent">
<androidx.appcompat.widget.Toolbar
android:id="@+id/tbCard"
android:layout_height="?attr/actionBarSize"
android:layout_width="match_parent"
android:minHeight="?attr/actionBarSize"
android:theme="@style/ThemeOverlay.AppCompat.ActionBar"
app:popupTheme="@style/ThemeOverlay.AppCompat.Light"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent"/>
</androidx.cardview.widget.CardView>

```

Donde menu_tarjeta contiene:

```

<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item android:id="@+id/action_borrar"
        android:title="@string/action_borrar"
        android:orderInCategory="100"
        app:showAsAction="never" />
</menu>

```

Ahora mismo la toolbar seguiría sin verse ya que no le hemos asignado ningún menú. Con la Toolbar anterior usábamos `setSupportActionBar()` donde el título y el menú lo tomó automáticamente. En esta ocasión la toolbar es independiente. Por ello, en el método `onCreate`, recuperamos ella referencia al control y lo asignamos con el método `inflateMenu()`. Por último solo nos quedaría asignar los oyentes:

```

val tbCard = findViewById<Toolbar>(R.id.tbCard)
tbCard.title = "tarjeta con menú"
tbCard.setOnMenuItemClickListener {
    when (it.itemId) {
        R.id.action_borrar -> {
            Log.i("ActionBar", "Acción borrar")
            true
        }
        else -> {
            true
        }
    }
}

```

```
}
tbCard.inflateMenu(R.menu.menu_tarjeta)
```

Y obtenemos como resultado:

Beneficios de Toolbar:

- No hay ninguna posición codificada en la aplicación.
- Diseño flexible y mucho más personalizable que ActionBar.
- Utiliza el último tema de Material de Android
- Proporciona compatibilidad con versiones anteriores hasta Android 2.1 (API 7).

Tabla de diferencias

AppBar/actionbar	Toolbar
Es un elemento de control o navegación dedicado presente en la parte superior de cada pantalla de la aplicación.	Es un ViewGroup que se puede colocar en cualquier lugar del diseño de la actividad.
Sólo puede haber una ActionBar en una actividad. Los desarrolladores tienen la opción de mostrar u ocultar la ActionBar.	Se pueden definir varios elementos de la barra de herramientas con una apariencia y un comportamiento completamente diferentes dentro de una sola actividad. Una barra de herramientas también se puede utilizar como barra de acciones.
Ofrece posibilidades de personalización. Los desarrolladores pueden agregar el logotipo/icono de la aplicación, elementos del menú de acciones, menú adicional, etc.	Se necesitan menos líneas de código para realizar la misma personalización en la barra de herramientas que es posible en la barra de acciones. Además, la barra de herramientas proporciona más flexibilidad y control a los desarrolladores mediante los cuales pueden animar los elementos o aplicar reacciones a los eventos de desplazamiento.
Las características agregadas en diferentes niveles de API no brindan compatibilidad con versiones anteriores.	Proporciona compatibilidad con versiones anteriores hasta API 7 (Android 2.1) .

ViewPager2

En la actualidad muchas aplicaciones Android cuentan con un patrón común de pantalla deslizable. Este patrón es utilizado para mostrar sus pantallas principales como diferentes

secciones en la aplicación.

ViewPager ha evolucionado mucho desde sus inicios, pero desde 2019, Google anunció ViewPager2 que sustituye a la versión anterior y corrige algunas de sus debilidades. Como algunos de los últimos componentes vistos, ViewPager está incluido en AndroidX y se compila sobre RecyclerView.

Ahora modificaremos nuestro layout principal para añadir el widget ViewPager2

```
<androidx.viewpager2.widget.ViewPager2
    android:id="@+id/viewpager"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Esto añade un widget ViewPager2 en el diseño con el id viewpager. ViewPager por sí solo es simplemente un ViewGroup. Para añadirle la funcionalidad hay que agregarle un adaptador de datos.

Para ello, debemos de crear una nueva clase kotlin que implemente FragmentStateAdapter

```
class ViewPagerAdaptador(activity: AppCompatActivity, var itemCount: Int) :
    FragmentStateAdapter(activity) {

    override fun getItemCount(): Int {
        return itemCount
    }

    override fun createFragment(position: Int): Fragment {
        return when(position) {
            0, 2, 4 -> AnimalesFragment()
            else -> PlantasFragment()
        }
    }
}
```

Nota: Se han de crear los fragmentos Animales() y Plantas()

A esta clase le hemos llamado ViewPagerAdaptador que recibe la referencia a la actividad y el número de fragmentos a mostrar.

Se sobrescriben dos métodos:

- getItemCount(): devuelve el número total de elementos en el adaptador
- createFragment(): que devuelve una instancia de fragmentación para la posición indicada por position

Ahora ya solo nos quedaría asignar nuestro adaptador a el ViewPager2

```
viewPager=findViewById(R.id.viewpager)
val adapta=ViewPagerAdaptador(this, 2)
viewPager.adapter = adapta
```

En estas líneas creamos una instancia de ViewPagerAdaptador y lo asignamos a nuestro ViewPager2

ViewPager2 viene con una opción para cambiar la orientación fácilmente. Esto es fácil porque ViewPager2 utiliza un RecyclerView que a su vez utiliza un LayoutManager para gestionar el posicionamiento de las vistas dentro de ella.

Por tanto, para cambiar el comportamiento de cómo se deslizan la páginas bastaría con cambiar la orientación:

```
viewPager.orientation=ViewPager2.ORTIENTATION_VERTICAL
```

Conectar una TabLayout con ViewPager2

Mostrar pestañas a la vez que usas las pantallas deslizable es una buena manera de hacerle saber a los usuarios en que posición están.

Android proporciona TabLayout. Un widget para mostrar las pestañas que forma parte del paquete Material.

En nuestra plantilla principal debemos de añadir dicho widget:

```
<com.google.android.material.tabs.TabLayout
    android:id="@+id/appbartabs"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:tabMode="scrollable"/>
```



```

<androidx.viewpager2.widget.ViewPager2

    android:id="@+id/viewpager"

    android:layout_width="match_parent"

    android:layout_height="0dp"

    android:layout_weight="1"/>

```

Para enlazar el TabLayout y ViewPager2 tienes que usar TabLayoutMediator. Esto sincroniza el ViewPager y TabLayout para cambiar la posición cuando uno se hace clic o se desliza.

```

adapta=ViewPagerAdaptador(this, 6)
viewPager.adapter = adapta
tabLayout=findViewById(R.id.appbartabs)

TabLayoutMediator(tabLayout, viewPager) { tab, position ->
    when (position) {
        0 -> {
            tab.text = "Primera"
        }
        1 -> {
            tab.text = "Segunda"
        }
        2 -> {
            tab.text = "Tercera"
        }
        3 -> {
            tab.text = "Cuarta"
        }
        4 -> {
            tab.text = "Quinta"
        }
        5 -> {
            tab.text = "Sexta"
        }
    }
}.attach()

```